Automi, Calcolabilità e Complessità

Alessio Marini, 2122855

Appunti presi durante il corso di **Automi, Calcolabilità e Complessità** nell'anno **2025/2026** del professore Daniele Venturi.
Gli appunti li scrivo principalmente per rendere il corso più comprensibile **a me** e anche per imparare il linguaggio Typst. Se li usate per studiare verificate sempre le informazioni 🙏🏻.

Contatti:
alem1105
marini.2122855@studenti.uniroma1.it

September 27, 2025

Contents

1. Automi . . . 3
   1.1. Linguaggi Regolari . . . 3
   1.2. Operazioni sui Linguaggi . . . 7
   1.3. Non Determinismo . . . 7
   1.4. Equivalenza tra DFA e NFA . . . 9
   1.5. Chiusura per Linguaggi Regolari . . . 12
   1.6. Espressioni Regolari . . . 16

1. Automi

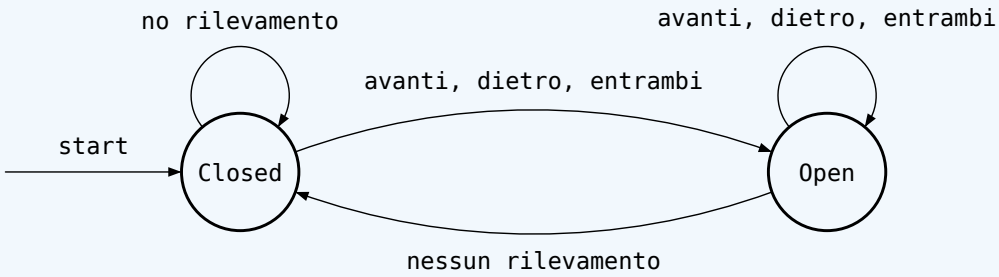
1.1. Linguaggi Regolari

Iniziamo introducendo in modo *informale* cos'è un **Automa**.
Un Automa è un modello matematico di calcolo che può elaborare informazioni e agire in modo automatico seguendo una serie di stati predefiniti.

Esistono diversi tipi di automi, il primo che vediamo si chiama **Automa a stati Finiti Deterministico (DFA)**, questo tipo di automa ha un **numero finito di stati** e processa gli input che riceve in modo **sequenziale** bit a bit.

Esempio - Sistema di controllo di una porta automatica

La porta ha dei sensori "avanti" e "dietro" per rilevare i movimenti ed aprire la porta.



Adesso però diamo una definizione più formale per i DFA.

Definizione - DFA

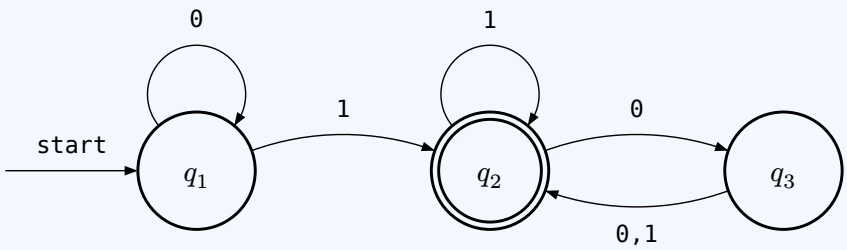
Un DFA è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

- Q: Insieme finito degli stati
- Σ: Insieme finito dei simboli in input (**alfabeto**)
- $\delta : Q \times \Sigma \to Q$: **Funzione di Transizione**, ci dice come avvengono i passaggi di stato
- q_0: Stato **iniziale**
- $F \subseteq Q$: Insieme degli **stati di accettazione**

Nei nostri diagrammi indicheremo gli stati di accettazione, o finali, con un *doppio bordo*. Inoltre diciamo che un automa "accetta" un input se termina in uno stato di accettazione, altrimenti diremo che lo "rifiuta".

Vediamo quindi un altro esempio di DFA.

Esempio - DFA di un linguaggio binario



Vediamo i vari elementi rispetto alla definizione che abbiamo dato prima:

- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$
- $q_0 = q_1$
- $F = \{q_2\}$

Mentre per la δ possiamo o rappresentare una tabella o in questo caso semplicemente guardare le frecce del diagramma. Questo DFA, ad esempio, *accetta* se riceve come input la stringa `w=11101`

Introduciamo altre definizioni che ci serviranno per definire in modo formale il concetto di **linguaggio accettato**.

In modo informale un linguaggio è l'insieme di tutte le stringhe rinosciute da un DFA, se ad esempio abbiamo il DFA M allora l'insieme delle stringhe riconosciute da M lo denotiamo con $L(M)$. Ad esempio il DFA dell'esempio precedente ha come linguaggio:

$$A = \{w \mid w \text{ contiene almeno un '1' e un numero pari di '0' segue l'ultimo '1'}\}$$

E possiamo dire quindi che $L(M) = A$ o anche "M riconosce A".

Per definire il linguaggio accettato dobbiamo modificare la funzione di transizione e introdurre la **funzione di transizione estesa**.

Definizione - Funzione di Transizione Estesa

La denotiamo con δ^* e ha lo stesso funzionamento della funzione di transizione classica, la differenza è che insieme allo stato accetta un'intera stringa e non un solo carattere.

- $\delta^* : Q \times \Sigma^* \to Q$
- $\delta^*(q, \varepsilon) = \delta(q, \varepsilon)$
- $\delta^*(q, ax) = \delta^*(\delta(q, a), x)$

Con Σ^* indichiamo l'insieme delle stringhe in input e $x \in \Sigma^*, a \in \Sigma$. Questo significa quindi che la funzione di transizione estesa processa intere stringhe ma sempre in modo sequenziale carattere per carattere.

Introduciamo anche il concetto di **configurazione**, ovvero lo stato in cui si trova il DFA, in particolare è definita come una tupla $(q, x) \in Q \times \Sigma^*$ e indica appunto lo stato in cui si trova l'automa e quello che gli resta da leggere. La configurazione iniziale è quella che si trova nello stato q_0 quindi ad esempio (q_0, x).

Adesso possiamo definire anche il **passo di computazione** ovvero una relazione binaria tra configurazioni che ci porta appunto da una configurazione ad un'altra. Si ha:

$$(p, ax) \vdash_M (q, x) \Leftrightarrow \delta(p, a) = q$$

Con $p, q \in Q, a \in \Sigma, x \in \Sigma^*$. Stiamo dicendo quindi che passiamo dalla configurazione (p, ax) alla configurazione (q, x) leggendo a (ci resta poi da leggere x) che è equivalente a dire che da p passiamo a q se leggiamo a.

Anche la relazione binaria $\vdash_M$ può essere estesa considerando la **chiusura riflessiva e transitiva** e la denotiamo con $\vdash_M^*$:

- $(q, x) \vdash_M^* (q, x)$, ovvero quando non legge nessun input.
- $(q, aby) \vdash_M (p, by) \wedge (p, by) \vdash_M (r, y) \Rightarrow (q, aby) \vdash_M^* (r, y)$, possiamo quindi raccogliere piú passi di configurazione.

Con $p, q, r \in Q; a, b \in \Sigma; y \in \Sigma^*$

Adesso possiamo finalmente utilizzare tutti questi concetti per formalizzare la definizione di **linguaggio accettato**.

Definizione - Linguaggio Accettato

Diciamo che $x \in \Sigma^*$ è accettato da $M = (Q, \Sigma, \delta, q_0, F)$ se

$$\delta^*(q_0, x) \in F \quad \text{oppure} \quad (q_0, x) \vdash_M^* (q, \varepsilon) \text{ con } q \in F$$

Possiamo definirlo anche come insieme ovvero $L(M) = \{x \in \Sigma^* : \delta^*(q_0, x) \in F\}$

Adesso che sappiamo cos'è un DFA e il suo linguaggio accettato, possiamo definire i **linguaggi regolari**.

Definizione - Linguaggio Regolare

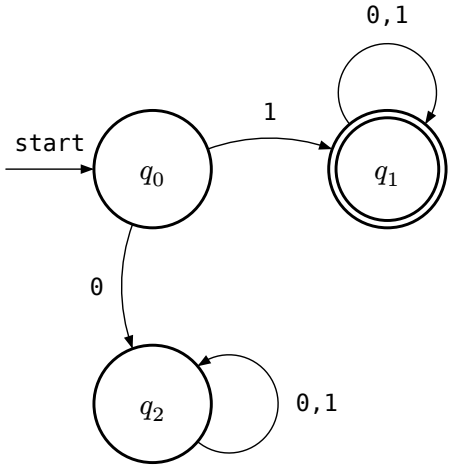
Un linguaggio è chiamato **regolare** se un DFA lo riconosce, indichamo con `REG` l'insieme dei linguaggi regolari:

$$\text{REG} = \{L \subseteq \Sigma^* \mid \exists \text{ DFA } M \text{ t.c. } L(M) = L\}$$

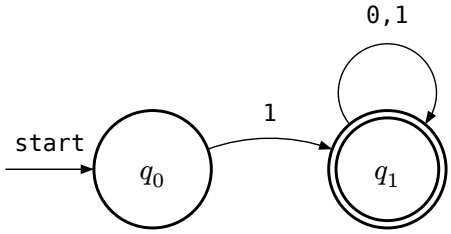
Ci interessa capire come costruire un DFA dato un certo linguaggio, ad esempio prendiamo il linguaggio:

$$L = \{x \in \{0, 1\}^* \mid x = 1 \| y, y \in \{0, 1\}^*\}$$

L'operatore $\|$ indica la concatenazione di stringhe, in questo caso quindi stiamo indicando tutte le stringhe che iniziano con 1. Proviamo a costruire un DFA A che riconosce il linguaggio:



Lo stato q_2 viene chiamato "stato pozzo", ovvero uno stato non di accettazione che non ha archi uscenti verso altri stati. Possiamo disegnare anche un DFA equivalente senza gli archi di q_2:



In questo caso se in q_0 riceviamo 0 semplicemente ignoramo l'input.

Adesso peró vogliamo dimostrare la correttezza del DFA appena costruito, ovvero dimostrare che il DFA accetta x **se e solo se** $x \in L$. Costruiamo una dimostrazione per induzione, per prima cosa però notiamo due cose nel nostro DFA:

- $\delta^*(q_1, u) = q_1 \ \forall u \in \{0,1\}^*$
- $\delta^*(q_2, u) = q_2 \ \forall u \in \{0,1\}^*$

Stiamo dicendo che se entriamo in q_1 o q_2 non andremo più in altri stati. Adesso iniziamo la dimostrazione per induzione:

- **Caso Base** - Abbiamo $|x| = 0$ ovvero $x = \varepsilon$ stringa vuota. Si ha che

$$\delta^*(q_0, \varepsilon) = \delta(q_0, \varepsilon) = q_0 \notin F$$

- **Ipotesi Induttiva** - Sia $n > 0$, supponiamo $|w| \leq n$. Si ha:

$$\delta^*(q_0, w) = \begin{cases} q_1 \text{ se } w \text{ inizia con } 1 \\ q_2 \text{ se } w \text{ inizia con } 0 \end{cases}$$

  Possiamo dirlo perchè abbiamo visto all'inizio che una volta entrati in q_1 o q_2 il DFA non cambia più stato.

- **Passo Induttivo** - Prendiamo x t.c. $|x| = n + 1$, possiamo scomporla in $x = au$ con $a \in \{0,1\}$ e $u \in \{0,1\}^*$. Otteniamo quindi

$$\delta^*(q_0, x) = \delta^*(q_0, au) = \delta^*\left(\underbrace{\delta(q_0, a)}_{\text{ha 2 soluzioni}}, u\right)$$

Le 2 soluzioni sono quelle che abbiamo visto nell'ipotesi induttiva, infatti:

$$\delta(q_0, a) = \begin{cases} q_2 \text{ se } a = 0 \\ q_1 \text{ se } a = 1 \end{cases}$$

Dato che abbiamo dimostrato che l'automa non uscirá mai da nessuno dei due stati e che entra sicuramente in uno dei due possiamo quindi dire che:

$$\delta^*(q_0, x) = q_1 \Leftrightarrow a = 1$$

1.2. Operazioni sui Linguaggi

I linguaggi non sono altro che insiemi di stringhe, questo significa che possiamo applicare delle operazioni su di essi, vediamole alcune:

- **Unione**: $L_1 \cup L_2 = \{x \in \Sigma^* \mid x \in L_1 \vee x \in L_2\}$
- **Intersezione**: $L_1 \cup L_2 = \{x \in \Sigma^* \mid x \in L_1 \wedge x \in L_2\}$
- **Complemento**: $\overline{L_1} = \{x \in \Sigma^* \mid x \notin L_1\}$
- **Concatenzaione**: $L_1 \circ L_2 = \{xy\} \mid x \in L_1 \wedge y \in L_2\}$ importante notare che non é commutativi quindi $L_1 \circ L_2 \neq L_2 \circ L_1$.

  Vediamo un esempio, prendiamo:

  - $\Sigma = \{a, b\}$
  - $L_1 = \{a, ab, ba\}$
  - $L_2 = \{ab, b\}$

  Otteniamo $L_1 \circ L_2 = \{aab, ab, abab, abb, baab, bab\}$
- **Potenza**: La definiamo in modo ricorsivo

  $$\text{per stringhe} \begin{cases} x^0 &= \varepsilon \\ x^{n+1} &= x^n x \end{cases} \text{; per linguaggi} \begin{cases} L^0 &= \{\varepsilon\} \\ L^{n+1} &= L^n \circ L \end{cases}$$

  Quindi ad esempio se prendiamo $L = \{a, ab, ba\}$ possiamo dire:

  $$L^2 = L \circ L = \{aa, aab, aba, aba, abab, abba, baa, baab, baba\}$$
- **Operatore** *: $L^* = \bigcup_{n \geq 0} L^n = \{\varepsilon\} \cup L^1 \cup L^2 \cup \ldots$ ovvero tutte le combinazioni di stringhe possibili. Ad esempio se $L = \{a, b\}$ abbiamo $L^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \ldots\}$

1.3. Non Determinismo

Per ora abbiamo visto come un DFA che si trova in uno stato quando processa un input farà un solo movimento verso un altro stato, oppure rimane nello stesso. Nel non determinismo invece abbiamo dei comportamenti diversi:

- Quando un automa si trova in uno stato e processa un input può andare in **diversi** stati **contemporaneamente**.
- Sono ammessi gli ε-archi, ovvero degli archi che non hanno bisogno di input per essere percorsi.

- L'automa a questo punto, visto che esplora più rami di computazione contemporaneamente, accetta se esiste almeno un ramo che accetta. Vedremo successivamente cosa significa "ramo di computazione".

A questo punto non parliamo più di DFA ma di **NFA** ovvero automi **non deterministici** a stati finiti.

Definizione - NFA

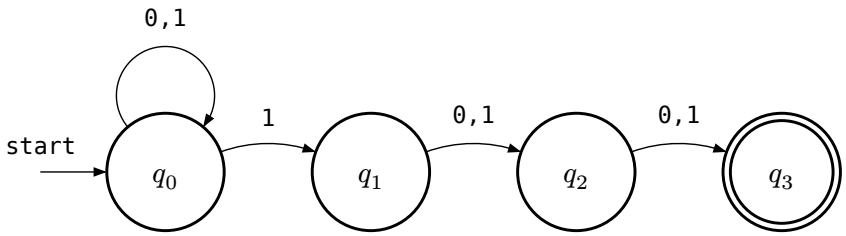
Un NFA è una tupla $(Q, \Sigma, \delta, q_0, F)$ dove tutti gli elementi sono come per i DFA ma abbiamo un cambio nella funzione di transizione. infatti questa diventa:

- $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$ con Σ_ε indichiamo $\Sigma \cup \{\varepsilon\}$, mentre con $\mathcal{P}(Q)$ l'insieme delle parti di Q ovvero l'insieme di tutti i possibili sottoinsiemi di Q

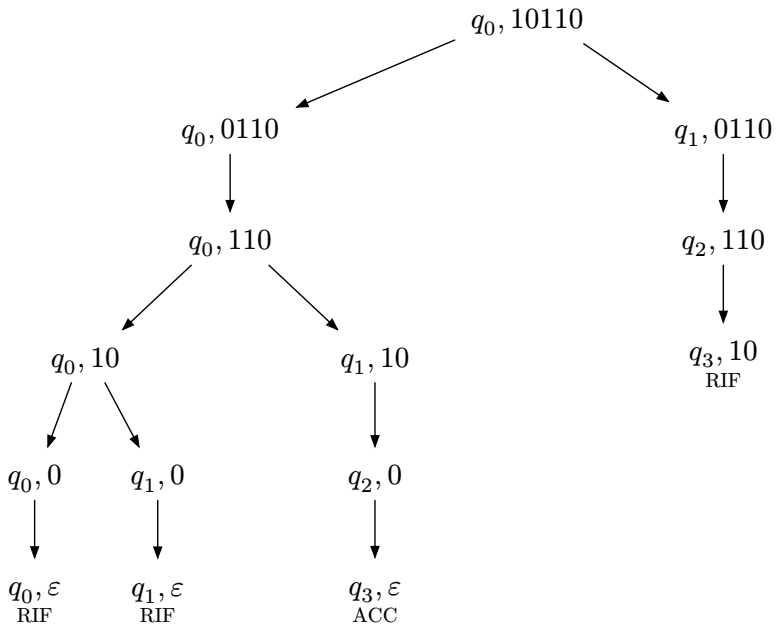
Per chiarire il concetto di "ramo di computazione" vediamo un esempio di NFA. Costruiamo un NFA che riconosce il linguaggio

$$L = \{x : x \in \{0,1\}^* \wedge x \text{ ha un '1' in terzultima posizione}\}$$

Costruiamo l'NFA:



Cosa succede quanto l'NFA computa la stringa `10110`? In un NFA quando uno stato permette più transizioni si creano più rami, uno per ogni strada presa, ad esempio con l'NFA appena costruito se inseriamo la stringa `10110` con il primo carattere `1` l'NFA andrà sia in q_0 che in q_1 e questo porta la creazione di due rami di computazione. L'automa accetta se esiste almeno un ramo fra questi che accetta. Per chiarezza vediamo quali rami si creano in questo esempio:

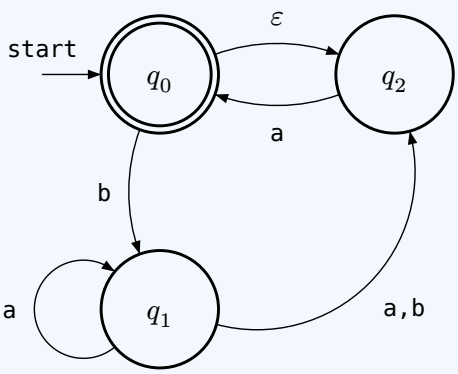


Per quanto riguarda gli ε-archi invece, se l'automa si trova in uno stato che ne possiede uno allora l'automa si duplica in due rami senza leggere nessun input:

- Un ramo rimane nella stessa configurazione di prima.
- L'altro ramo segue l'ε-arco.

Vediamo un altro esempio per chiarire anche questo concetto.

Esempio - NFA con ε-archi



Vediamo cosa succede con la computazione della stringa `babba`

$$q_0, babba \to q_1, abba \to q_2, bba \text{ RIF}$$
$$q_0, babba \to q_2, babba \text{ RIF}$$

Dobbiamo adesso rivedere i concetti di **configurazione, computazione ed accettazione** in un NFA:

- Dato un NFA N, indichiamo come configurazione una coppia $(q, x) \in Q, \Sigma_\varepsilon^*$
- Avremo

$$(p, ax) \vdash_N (q, x) \Leftrightarrow q \in \delta(p, a) \quad \text{con } x \in \Sigma_\varepsilon^*; a \in \Sigma_\varepsilon; p, q \in \Sigma$$

  Da notare che q, p non sono più degli stati singoli ma un insieme di stati infatti $q, p \in \mathcal{P}(Q)$, ovviamente possono anche contenere un singolo stato ma saranno comunque insiemi di stati.

- Un NFA N accetta

$$w \in \Sigma_\varepsilon^* \Leftrightarrow \exists q \in F \text{ t.c. } (q_0, w) \vdash_N^* (q, \varepsilon)$$

1.4. Equivalenza tra DFA e NFA

Dimostriamo come in realtá le due classi di automi sono equivalenti. Per prima cosa definiamo in modo formale le due classi:

- $\mathcal{L}(\text{DFA}) = \text{REG}$, quindi la classe dei linguaggi regolari, riconosciuti da almeno un DFA.
- $\mathcal{L}(\text{NFA}) = \{L | \exists \text{ NFA } N \text{ t.c. } L(N) = L\}$, la classe dei linguaggi riconosciuti da almeno un NFA.

Teorema

REG $:= \mathcal{L}(\text{DFA}) = \mathcal{L}(\text{NFA})$

Dimostrazione - Formalmente vogliamo dimostrare $\mathcal{L}(\text{DFA}) \subseteq \mathcal{L}(\text{NFA})$ e $\mathcal{L}(\text{NFA}) \subseteq \mathcal{L}(\text{DFA})$

1. **Prima implicazione** $\mathcal{L}(\text{DFA}) \subseteq \mathcal{L}(\text{NFA})$

   Dato $L \in \mathcal{L}(\text{DFA})$ e $D := (Q, \Sigma, \delta, q_0, F)$ t.c. $L = L(D)$. Notiamo che il concetto di NFA, in realtà, non è altro che una generalizzazione di un DFA, infatti se prendiamo un qualsiasi DFA possiamo dire che è anche un NFA, quindi D è un NFA e $L \in \mathcal{L}(\text{NFA})$.

2. **Seconda implicazione** $\mathcal{L}(\text{NFA}) \subseteq \mathcal{L}(\text{DFA})$

   Per dimostrare questa implicazione prendiamo un NFA e costruiamo un DFA che lo simula. Sia
   - $N = \{Q, \Sigma, \delta, q_0, F\}$ l'NFA t.c. $L(N) = A$

   costruiamo il DFA
   - $M = \{Q_M, \Sigma, \delta_M, q_0^M, F_M\}$ t.c. $L(M) = A$

   Prima facciamo una costruzione senza considerare gli ε-archi:

   - $Q_M = \mathcal{P}(Q)$ - Ogni stato di M è un insieme di stati N
   - $\delta_M(R, a) = \bigcup_{r \in R} \delta(r, a)$

     Dove $R \in Q_M, a \in \Sigma, \delta(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ con } r \in R\}$, stiamo dicendo che la funzione di transizione si applica su un insieme di stati e restituisce un altro insieme di stati, precisamente quelli che si ottengono applicando la funzione di transizione su tutti gli elementi dell'insieme in input.
   - $q_0^M = \{q_0\}$
   - $F_M = \{R \in Q_M \mid R \cap F \neq \emptyset\}$ ovvero quegli insiemi di stati che hanno al loro interno almeno uno stato accettante del DFA.

   A questo punto introduciamo gli ε-archi e per farlo definiamo

   $$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso } 0 \text{ o più } \varepsilon - \text{archi}\}$$

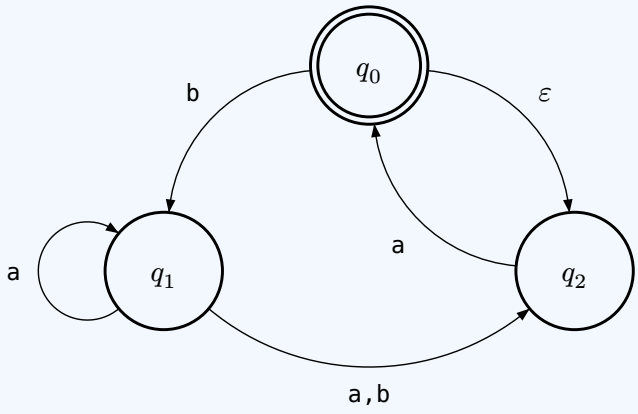
   Scriviamo 0 o più perchè in questo modo includiamo anche il nodo su cui ci troviamo. Con questa definizione possiamo modificare la funzione di transizione del DFA e lo stato iniziale del DFA in:

   - $\delta_M(R, a) = \{q \in Q \mid q \in E(\delta(r, a)), \exists r \in R\}$. In questo modo includiamo sia gli archi visti con la definizione di prima ma anche quelli che raggiungiamo tramite gli ε-archi.
   - $q_0^M = \{q_0\}$

Vediamo un esempio di trasformazione da NFA a DFA per chiarire i concetti.

Esempio - Da NFA a DFA

Prendiamo questo NFA come esempio, ha come linguaggio $\Sigma = \{a, b\}$



Iniziamo a dare le prime definizioni:

- $Q_D = \left\{q_\emptyset, q_{\{0\}}, q_{\{1\}}, q_{\{2\}}, q_{\{0,1\}}, q_{\{0,2\}}, q_{\{1,2\}}, q_{\{0,1,2\}}\right\}$. Con la notazione $q_{\{0,1\}}$ indichiamo l'insieme di stati $\{q_0, q_1\}$
- $q_0^D = E\left(q_{\{0\}}\right) = q_{\{0,2\}}$. Infatti da q_0 ci muoviamo con un ε-arco in q_2 ma manteniamo anche il ramo che si trova in q_0.
- $F_D = \left\{q_{\{0\}}, q_{\{0,1\}}, q_{\{0,2\}}, q_{\{0,1,2\}}\right\}$. Ovvero tutti gli stati che contengono lo stato q_0

Adesso ci manca da definire la funzione di transizione, è un passaggio lungo quindi vediamo solo alcuni casi.

Per lo stato $q_{\{0\}}$ abbiamo:

- $\delta_D\left(q_{\{0\}}, a\right) = E(\delta(q_0, a)) = q_{\{0,2\}}$
- $\delta_D\left(q_{\{0\}}, b\right) = E(\delta(q_0, b)) = q_{\{1,2\}}$

Per lo stato $q_{\{1\}}$:

- $\delta_D\left(q_{\{1\}}, a\right) = E(\delta(q_1, a)) = q_{\{1,2\}}$
- $\delta_D\left(q_{\{1\}}, b\right) = E(\delta(q_1, b)) = q_{\{2\}}$

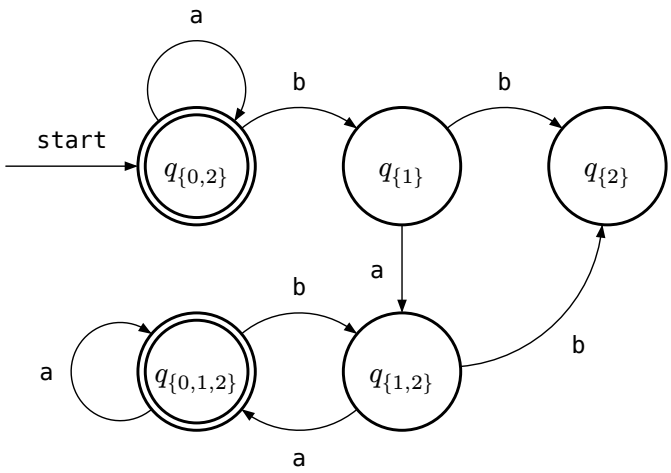
...

Per lo stato $q_{\{0,1\}}$:

- $\delta_D\left(q_{\{0,1\}}, a\right) = E(\delta(q_0, a)) \cup E(\delta(q_1, a)) = q_{\{0,2\}} \cup q_{\{1,2\}} = q_{\{0,1,2\}}$
- ...

...

Basandoci sull'esempio di prima possiamo anche provare a costruire il DFA che abbiamo soltanto definito. (semplificando anche alcuni nodi mai raggiunti)



Da adesso in poi quindi non ci importa più distinguere NFA e DFA dato che abbiamo visto essere equivalenti, possiamo usare benissimo entrambi in qualsiasi situazione, ovviamente in alcuni casi potrebbe essere più vantaggioso un determinato tipo.

1.5. Chiusura per Linguaggi Regolari

I linguaggi regolari sono **chiusi** rispetto alle loro operazione? Ovvero, se applichiamo un'operazione a due linguaggi regolari otteniamo come risultato un altro linguaggio regolare?

Teorema

L'insieme REG è chiuso per unione. Ovvero $L_1, L_2 \in \text{REG} \Rightarrow L_1 \cup L_2 \in \text{REG}$

Dimostrazione - Possiamo svolgere la dimostrazione in due modi:

1. Senza NFA

   Per dimostrare il teorema utilizzando i DFA dobbiamo costruire un automa M che riconosce il linguaggio $L_1 \cup L_2$. Per farlo dobbiamo fare in modo che M simuli il comportamento di altri due automi M_1, M_2 tali che $L(M_1) = L_1$ e $L(M_2) = L_2$, e che accetti soltanto se almeno una delle due simulazioni accetta.

   Dati

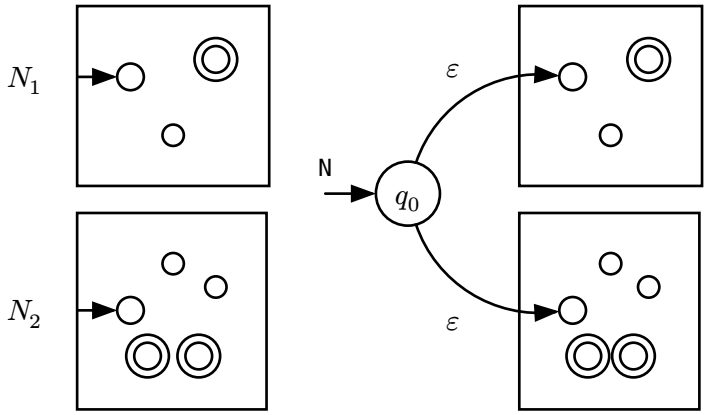
   - $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ t.c. $L(M_1) = L_1$
   - $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $L(M_2) = L_2$

   Costruiamo quindi $M = (Q, \Sigma, \delta, q_0, F)$ dove:

   - $Q = Q_1 \times Q_2 = \{(r_1, r_2) \mid r_1 \in Q_1 \wedge r_2 \in Q_2\}$
   - $\delta : Q \times \Sigma \to Q$. Ad esempio $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta(r_2, a))$
   - $q_0 = (q_1, q_2)$
   - $F = \{(r_1, r_2) \mid r_1 \in F_1 \vee r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

2. Con NFA

   In questo caso la dimostrazione è molto semplice, infatti ci basiamo sempre sull'idea di prima di usare due simulazioni N_1, N_2 per i due linguaggi ma le inglobiamo in un NFA N che avrá come nodo iniziale un nuovo nodo collegato ai nodi iniziali delle simulazioni tramite ε-archi:



Più formalmente, dati

- $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ t.c. $L(N_1) = L_1$
- $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $L(N_2) = L_2$

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ dove:

- $Q = \{q_0\} \cup Q_1 \cup Q_2$
- $F = F_1 \cup F_2$
- $\delta : Q \times \Sigma \to Q$ dove

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0, a = \varepsilon \\ \emptyset & q = q_0, a \neq \varepsilon \end{cases}$$

Teorema

REG è chiuso per complemento. Ovvero $\forall L \in \text{REG}, \overline{L} \in \text{REG}$

Dimostrazione - Costruisco un automa $\overline{L}$ che accetta tutto tranne le stringhe accettate da L. Per farlo ci basta avere come stati finali il complemento dell'insieme degli stati finali.

Dato $L = (Q, \Sigma, \delta, q_0, F)$ t.c. $L(L) = A$ costruisco $\overline{L} = (Q, \Sigma, \delta, q_0, Q - F)$

Teorema

REG è chiuso per intersezione. Ovvero $\forall L_1, L_2 \in \text{REG}, L_1 \cap L_2 \in \text{REG}$

Dimostrazione - Dati $D_1 = (Q, \Sigma, \delta_1, q_1, F_1)$ t.c. $\mathcal{L}(D_1) = L_1$ e $D_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $\mathcal{L}(D_2) = L_2$ costruisco $M = (Q, \Sigma, \delta, q_0, F)$ t.c.:

- $Q = Q_1 \times Q_2$
- $F = F_1 \times F_2$
- $q_0 = (q_1, q_2)$
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$

Teorema

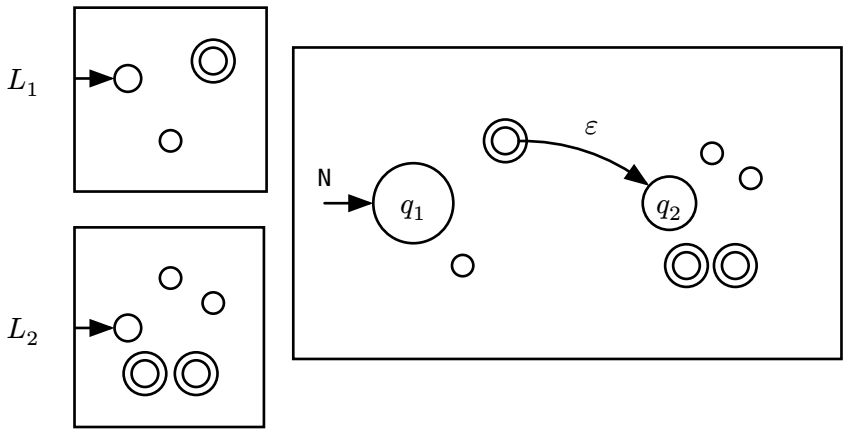
REG è chiuso per concatenazione. $\forall L_1, L_2 \in \text{REG}, L_1 \circ L_2 \in \text{REG}$

Dati $D_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ t.c. $\mathcal{L}(D_1) = L_1$ e $D_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $\mathcal{L}(D_2) = L_2$ costruisco $N = (Q, \Sigma, \delta, q_0, F)$ t.c.:

- $Q = Q_1 \cup Q_2$
- $q_0 = q_1$
- $F = F_2$
- $\forall q \in Q, a \in \Sigma$ si ha che

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 - F_1 \\ \delta_1(q,a) & q \in F_1 \wedge a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1 \wedge a = \varepsilon \\ \delta_2(q,a) & q \in Q_2 \end{cases}$$

Graficamente ci basta prendere il primo automa L_1 e collegare i suoi stati finali allo stato iniziale di L_2 tramite ε-archi:



Da questa dimostrazione possiamo anche dire che REG è chiuso per potenza, dato che la potenza si esprime in funzione della concatenazione. *Es* $L^2 = L \circ L$

Corollario

REG è chiuso per l'operatore potenza. $\forall L \in \text{REG}, n \in \mathbb{N}, L^n \in \text{REG}$.

Teorema

REG è chiuso per l'operatore $*$. $\forall L \in \text{REG}, L^* \in \text{REG}$.

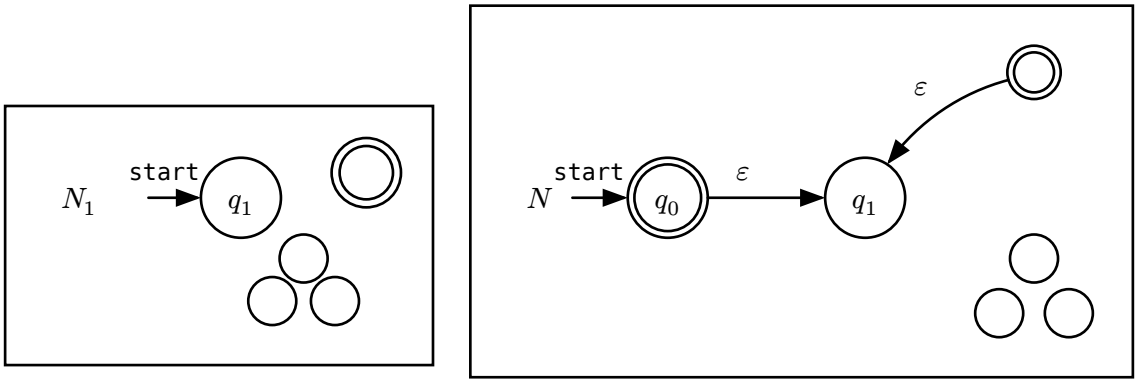
Dimostrazione - Dato $L \in \text{REG}$, sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F)$ l'NFA t.c. $L(N_1) = L$ costruisco $N = (Q, \Sigma, \delta, q_0, F)$ NFA t.c.:

- $q_0 \neq q_1$ è un nuovo stato iniziale
- $Q = Q_1 \cup \{q_0\}$
- $F = F_1 \cup \{q_0\}$. Dobbiamo aggiungere anche il nuovo stato iniziale dato che l'operatore $*$ comprende anche la stringa vuota ε.

- $\forall q \in Q, a \in \Sigma$ si ha che

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q - F \\ \delta_1(q,a) & q \in F \wedge a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F \wedge a = \varepsilon \\ \{q_1\} & q = q_0 \wedge a = \varepsilon \\ \emptyset & q = q_0 \wedge a \neq \varepsilon \end{cases}$$

Questo nuovo automa accetta tutto quello che accetta N_1 ma ogni volta che accetta torna indietro con un ε-arco per provare ad accettare un'altra stringa accettata sempre da N_1. Proviamo a rappresentare questo automa:



Come prima, grazie a questa dimostrazione possiamo anche dire che REG è chiuso per +. L'operatore + é come l'operatore $*$ ma esclude la potenza 0, posiamo definirlo quindi come $L^+ = L^1 \cup L^2 \cup \ldots$

Notazione

L^+ equivale quindi a scrivere LL^*, ovvero almeno una concatenazione di L.

Corollario

REG è chiuso per $+$. La dimostrazione è uguale a $*$ ma ci basta escludere q_0 da F.

1.6. Espressioni Regolari

Le espressioni regolari sono come le espressioni matematiche ma invece di rappresentare numeri, rappresentano linguaggi. Diamo una definizione formale.

Definizione - Espressione Regolare

Sia Σ un alfabeto. Un'espressione regolare su Σ che indichiamo con $re(\Sigma)$ è definita ricorsivamente come segue:

$$\text{caso base: } \begin{cases} \emptyset & \in re(\Sigma) \\ \varepsilon & \in re(\Sigma) \\ a & \in re(\Sigma), \forall a \in \Sigma \end{cases}$$

$$\text{caso induttivo: } \begin{cases} R_1 \cup R_2 & R_1, R_2 \in re(\Sigma) \\ R_1 \circ R_2 & R_1, R_2 \in re(\Sigma) \\ (R_1)^* & R_1 \in re(\Sigma) \end{cases}$$

Dove R_i sono espressioni regolari.

Stiamo quindi dicendo che, preso un alfabeto, i suoi singoli caratteri, la stringa vuota e l'insieme vuoto appartengono all'espressione regolare che descrive questo alfabeto. L'unione, l'intersezione o la $*$ di espressioni regolari su questo alfabeto sono comunque espressioni regolari dell'alfabeto.

Osservazione - Ogni espressione regolare $R \in re(\Sigma)$ ha associato un linguaggio $L(R)$:

$$\text{caso base: } \begin{cases} L(R) = \emptyset & \text{se } R = \emptyset \\ L(R) = \{\varepsilon\} & \text{se } R = \varepsilon \\ L(R) = \{a\} & \text{se } R = a \end{cases}$$

$$\text{caso induttivo: } \begin{cases} L(R) = L(R_1) \cup L(R_2) & \text{se } R = R_1 \cup R_2 \\ L(R) = L(R_1) \circ L(R_2) & \text{se } R = R_1 \circ R_2 \\ L(R) = (L(R_1))^* & \text{se } R = (R_1)^* \end{cases}$$

Vediamo qualche esempio di espressione regolare.

Esempi

- $0^*10^* = \{w \mid w \text{ contiene un solo } 1\}$
- $(0 \cup 1) = \{0\} \cup \{1\} = \{0, 1\}$
- $(\Sigma\Sigma)^* = \{w \mid w \text{ ha lunghezza pari}\}$. Prima concatena 2 simboli qualsiasi dell'alfabeto e poi li eleva, quindi avremo semrpe $2 * n$ simboli

Teorema

Un linguaggio è regolare **se e solo se** esiste un'espressione regolare che lo descrive, ovvero:

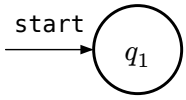
$$\text{REG} \equiv \mathcal{L}(re)$$

Dimostrazione - Dimostriamo il teorema tramite due lemmi.

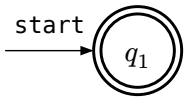
1. **Lemma** $\mathcal{L}(re) \subseteq \text{REG}$

Data un'espressione regolare R, costruisco un NFA/DFA N_R t.c. $L(N_R) = L(R)$. Per convertire R in un NFA consideriamo i casi della definizione ricorsiva di R. Prima vediamo i 3 casi base e poi i 3 induttivi.

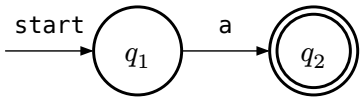
1. $R = \emptyset$. È un NFA che non accetta niente.
   - $N_R = \{\{q_1\}, \Sigma, \delta, q_1, \emptyset\}$
   - $\delta(q_1, b) = \emptyset, \forall b \in \Sigma$



2. $R = \varepsilon$. È un NFA che accetta soltanto la stringa vuota.
   - $N_R = \{\{q_1\}, \Sigma, \delta, q_1, \{q_1\}\}$
   - $\delta(q_1, b) = \emptyset, \forall b \in \Sigma$



3. $R = a, a \in \Sigma$. È un NFA che accetta soltanto il carattere a.
   - $N_R = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$
   - $\delta(q_1, a) = q_2$
   - $\delta(q, b) = \emptyset, b \neq a$



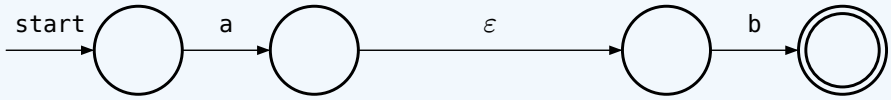
Passiamo adesso ai 3 casi ricorsivi.

4.5.6. $R = R_1 \cup R_2; R = R_1 \circ R_2; R = (R_1)^*$. Per costruire i casi induttivi ci basta utilizzare le chiusure dei linguaggi regolari. Ad esempio se dobbiamo costruire $R = R_1 \cup R_2$ ci basta prima costruire i due automi per R_1, R_2 e poi utilizzare la chiusura come visto nelle dimostrazioni precedenti.

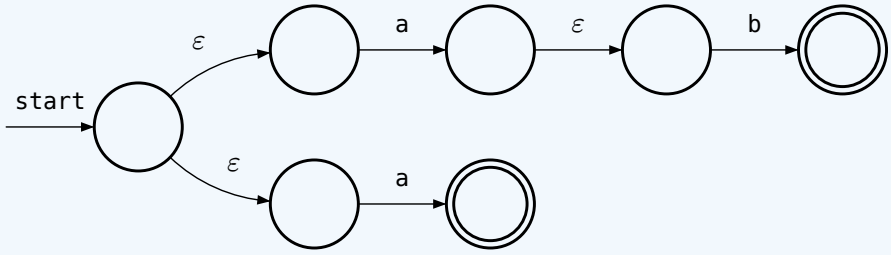
Esempio

Convertiamo l'espressione regolare $(ab \cup a)^*$ in un NFA: Per prima cosa costruiamo gli automi per riconoscere i caratter a e b:

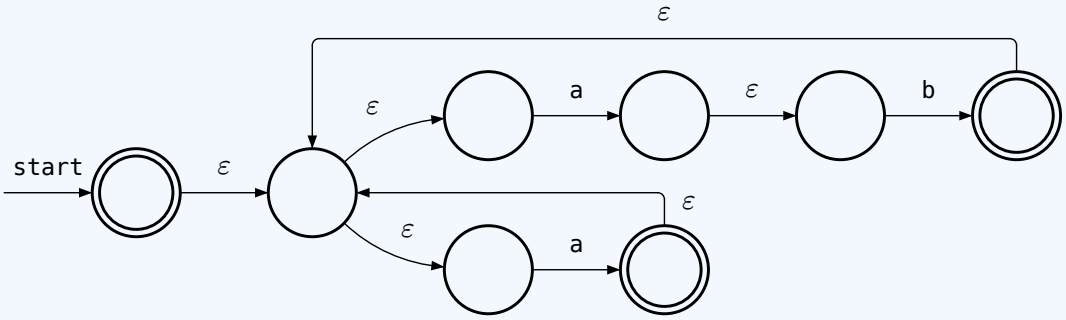
Poi costruiamo quello per riconoscere ab usando appunto la concatenazione:



Adesso usiamo l'unione per costruire l'NFA che riconosce $ab \cup a$:



Possiamo finalmente usare la chiusura per $*$ e costruire l'NFA:



2. **Lemma** $\text{REG} \subseteq \mathcal{L}(re)$. Adesso dobbiamo mostrare come passare da un linguaggio regolare ad un'espressione regolare. Per farlo dobbiamo imparare a convertire gli NFA in espressioni regolari.

   Per fare questo ci serve il concetto di **NFA Generalizzato (GNFA)**:

- Le etichette degli archi sono espressioni regolari
- Lo stato iniziale ha solo archi uscenti verso ogni altro stato
- C'è un solo stato finale, che ha solo archi entranti da altri stati ed è diverso dallo stato iniziale
- Eccetto gli stati iniziale e finale, presi due stati qualsiasi (anche lo stesso) esiste un arco tra di essi.

Per prima cosa convertiamo l'NFA in un GNFA tramite il seguente algoritmo:

- Se lo stato iniziale ha archi entranti, aggiungere un nuovo stato iniziale con un ε-arco al vecchio stato iniziale.
- Se c'è più di uno stato accettante o esiste uno stato accettante con archi uscenti, aggiungere un nuovo stato finale con ε-archi provenienti da ogni $q \in F$.

- Se qualche arco ha più di un input, usare come input per quell'arco l'unione tra gli input precedenti.
- Aggiungere archi con input $\emptyset$ tra le coppie di stati non unite da archi.

Possiamo anche fornire una definizione formale di **GNFA**.

Definizione - GNFA

Un GNFA è una tupla $\left(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}}\right)$ dove gli elementi sono definiti come negli NFA ad eccezione della funzione di transizione, infatti:

$$\delta : \left(Q \setminus \left\{q_{\text{accept}}\right\}\right) \times \left(Q \setminus \left\{q_{\text{start}}\right\}\right) \to \mathcal{R} = re(\Sigma)$$

Conversione da GNFA a Espressione Regolare - Adesso possiamo convertire il GNFA in espressione regolare. Definiamo la funzione `CONVERT(G)` che prende in input un GNFA e restituisce un'espressione regolare:

1. Sia $k = \#\text{stati di } G$
2. Se $k = 2$ allora G ha solo i due stati q_{start} e q_{accept} e un singolo arco con etichetta $R \in \mathcal{R}$, l'output sarà quindi R.
3. Se $k > 2$ scegliamo un $q_{\text{rip}} \in Q$ diverso da q_{start} e q_{accept}. Definiamo $G' = \left(Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}}\right)$ dove:
   - $Q' = Q \setminus \left\{q_{\text{rip}}\right\}$
   - $\delta' : Q' \setminus \{q_{\text{acc}}\} \times Q' \setminus \{q_{\text{start}}\} \to \mathcal{R}$

   Quindi prendiamo uno stato q_{rip} a caso e lo rimuoviamo. Come si rimuove? Vediamo un esempio: